

# Wykrywanie fake news – porównanie modeli językowych

## LIAR dataset: RoBERTa fine-tuning vs. GPT-4o zero-shot i few-shot

Czerwiec 2026

## 1 Wprowadzenie

Celem projektu jest porównanie różnych podejść do automatycznego wykrywania fałszywych informacji w wypowiedziach politycznych. Zadanie realizowane jest na zbiorze LIAR (Wang, 2017), zawierającym 12 791 krótkich wypowiedzi oznaczonych jednym z sześciu poziomów prawdziwości przez serwis fact-checkingowy PolitiFact. Porównujemy dwa zasadniczo różne paradygmaty: *fine-tuning* pretrenowanego enkodera (RoBERTa) oraz *prompting* dużego modelu generatywnego (GPT-4o-mini i GPT-4o) w trybach zero-shot i few-shot.

Zadanie rozważamy w dwóch wariantach:

- **Klasyfikacja binarna (2kl)** – etykiety **fake** (oryginalnie **pants-fire**, **false**, **barely-true**) i **real** (**half-true**, **mostly-true**, **true**).
- **Klasyfikacja sześcioklasowa (6kl)** – pełne etykiety LIAR.

## 2 Zbiór danych

LIAR składa się z 10 240 wypowiedzi treningowych, 1 284 walidacyjnych i 1 267 testowych. Każda wypowiedź zawiera: treść, mówcę, partię polityczną, kontekst wypowiedzi, temat oraz historię wcześniejszych ocen prawdziwości danego mówcy.

Dziesięć najczęściej omawianych tematów w zbiorze danych to: gospodarka, opieka zdrowotna, podatki, budżet federalny, edukacja, zatrudnienie, budżet stanowy, biografie kandydatów, wybory i imigracja<sup>1</sup>.

### 2.1 Rozkład klas

W wariancie binarnym zbiór jest umiarkowanie niebilansowany (5 752 **real** vs 4 488 **fake** w zbiorze treningowym – 56% vs 44%). W wariancie sześcioklasowym dysproporcje są wyraźniejsze: najrzadsza klasa **pants-fire** stanowi 8% danych, najczęstsza **half-true** – 21%. Brak zbalansowania jest istotny dla wyboru funkcji straty (sekcja 3).

### 2.2 Przygotowanie tekstu

Każda wypowiedź reprezentowana jest jako tekst łączący wiele cech:

```
{speaker} ({party}, past lies: {n}, past truths: {m}):  
{statement} Subject: {subject} Context: {context}
```

---

<sup>1</sup>Źródło: <https://aclanthology.org/P17-2067/> (Wang 2017).

Tak skonstruowany ciąg daje modelowi dostęp nie tylko do treści wypowiedzi, ale również do meta-danych mówcy. Tekst jest tokenizowany tokenizatorem RoBERTa z długością maksymalną 128 tokenów – 99,96% wypowiedzi mieści się w tym limicie, mediana długości wynosi 22 tokeny.

BERT i RoBERTa jako modele pretrenowane przyjmują wejście wyłącznie w postaci wektorów liczb całkowitych, gdzie każda wartość reprezentuje token. Konwersję tę wykonują dedykowane tokenizatory (osobne dla BERT-a i RoBERTy), które dodają tokeny specjalne początku i końca sekwencji, dzielą tekst na tokeny (słowa, sub-słowa lub znaki) i przypisują każdemu unikalne ID, tworząc wymagany wektor liczb całkowitych.

## 3 Metody

### 3.1 RoBERTa fine-tuning

Wybraliśmy model `roberta-large` (355M parametrów) zamiast `bert-base` z następujących powodów (Liu et al., 2019):

1. **Dynamiczne maskowanie** – RoBERTa losuje maskowane tokeny przy każdym przejściu, BERT stosuje statyczne maskowanie. Lepsze pokrycie kontekstu w trakcie pretreningu.
2. **Większy korpus** – 160 GB tekstu (Wikipedia, książki, CommonCrawl, OpenWebText, news) zamiast 16 GB BERT-a. Bogatsza wiedza ogólna, szczególnie istotna w domenie newsów politycznych.
3. **Brak zadania NSP** (Next Sentence Prediction) – udowodniono, że NSP nie pomaga w klasyfikacji pojedynczych zdań, a wręcz pogarsza wyniki.
4. **Lepsze hiperparametry pretreningu** – większe batche (8K), dłuższy trening.

BERT był pretrenowany na BooksCorpus i angielskiej Wikipedii, natomiast RoBERTa na BooksCorpus, CC-News, OpenWebText i Stories – znacznie większym i bardziej zróżnicowanym korpusie.

RoBERTa jest modelem typu *encoder-only* – zawiera wyłącznie część enkoderową architektury Transformer, dzięki czemu produkuje gęsty wektor reprezentacji wejścia (a nie generuje tekstu), co czyni ją naturalnym wyborem do zadań klasyfikacji.

Model jest trenowany z głową klasyfikacji (Dropout → Linear → tanh → Dropout → Linear) podpinaną do reprezentacji tokenu [CLS].

#### 3.1.1 Hiperparametry treningu

- Liczba epok: 5
- Optymalizator: AdamW,  $lr = 1 \cdot 10^{-5}$ , `weight_decay = 0,01`
- Batch size: 16 (A100) lub 8 (T4)
- Learning rate schedule: liniowy warm-up przez 1 epokę, następnie liniowy decay do zera (`get_linear_schedule_with_warmup`)
- Gradient clipping: 1,0
- Selekcja epoki: najlepszy walidacyjny macro-F1 (model najlepszej epoki zapisywany jest na dysk)

### 3.1.2 Wagi klas

Ze względu na niezbilansowanie klas (zwłaszcza w wariancie 6kl) stosujemy ważoną funkcję straty Cross-Entropy:

$$w_c = \frac{N}{C \cdot n_c},$$

gdzie  $N$  to liczba wszystkich przykładów,  $C$  to liczba klas, a  $n_c$  to liczba przykładów klasy  $c$ . Wagi te dla 2kl wynoszą  $w_{\text{fake}} = 1,14$ ,  $w_{\text{real}} = 0,89$ , a dla 6kl rozciągają się od ok. 0,81 dla **half-true** do ok. 2,03 dla **pants-fire**. Mechanizm ten przeciwdziała tendencji modelu do ignorowania klas mniejszościowych.

### 3.2 GPT prompting – zero-shot i few-shot

Drugie podejście wykorzystuje modele GPT (**gpt-4o-mini**, **gpt-4o**) przez API OpenAI w dwóch trybach:

- **Zero-shot** – model otrzymuje wyłącznie opis zadania i jedną wypowiedź do oceny.
- **Few-shot** – przed wypowiedzią testową prompt zawiera  $K$  przykładów z każdej klasy ze zbioru treningowego.

Przykłady few-shot dobierane są przez *nearest-neighbor retrieval* oparty na cosine similarity reprezentacji TF-IDF (1- i 2-gramy). Dla każdej klasy wybieramy  $K_{\text{per\_class}} = 3$  najbardziej podobne wypowiedzi do zapytania (łącznie 6 przykładów dla 2kl i 18 dla 6kl).

### 3.3 Metryki

Główną metryką jest **macro-F1** – średnia F1 obliczona niezależnie dla każdej klasy, bez ważenia liczebnością. Wybór ten jest podyktowany niezbilansowaniem danych: accuracy oraz weighted-F1 byłyby dominowane przez klasy większościowe i nie odzwierciedlałyby umiejętności modelu na klasach rzadkich (krytyczne dla 6kl).

Dodatkowo raportujemy accuracy, precision i recall per klasa oraz macierze pomyłek.

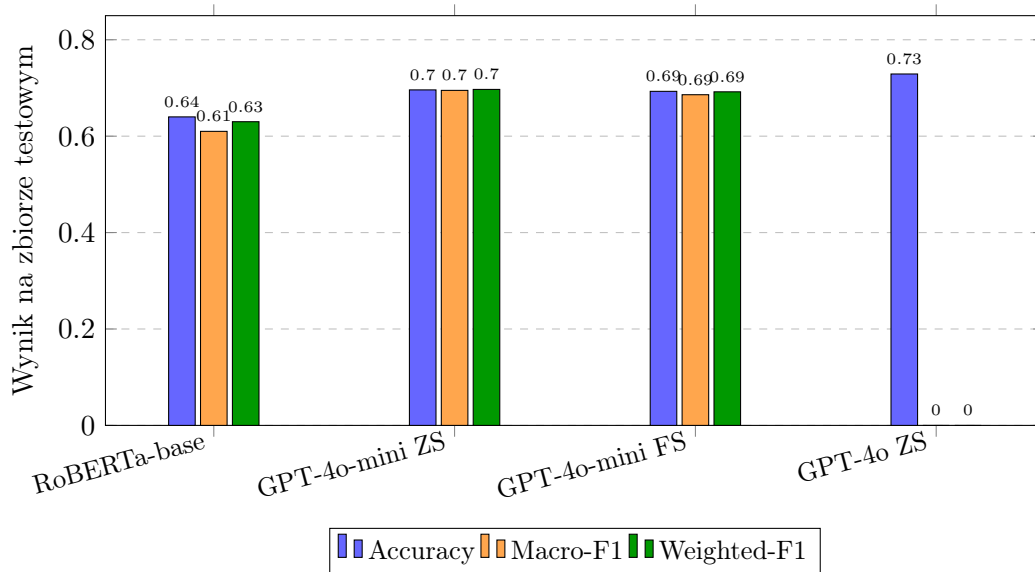
## 4 Wyniki

Rysunek 1 prezentuje porównanie modeli na wariancie binarnym, a rysunek 2 – na wariancie sześcioklasowym.

### 4.1 Zestawienie fine-tuningu (encoder)

Tabela 1: Porównanie modeli typu encoder fine-tuned na LIAR (zbiór testowy). BERT-base i RoBERTa-base trenowane były tylko w wariancie 2-klasowym.

| Model                            | 2kl Acc      | 2kl Macro-F1 | 6kl Acc      | 6kl Macro-F1 |
|----------------------------------|--------------|--------------|--------------|--------------|
| BERT-base-uncased                | 0,628        | 0,604        | —            | —            |
| RoBERTa-base                     | 0,640        | 0,622        | —            | —            |
| <b>RoBERTa-large + Optuna</b>    | <b>0,731</b> | <b>0,726</b> | <b>0,357</b> | <b>0,359</b> |
| <b>RoBERTa-large + LIAR-PLUS</b> | —            | —            | <b>0,383</b> | <b>0,380</b> |



Rysunek 1: Porównanie modeli na zbiorze testowym LIAR – wariant binarny. Wartość 0 oznacza brak pomiaru. ZS = zero-shot, FS = few-shot (kNN).

Tabela 2: Najlepsze wyniki z każdej kategorii modeli (macro-F1 na zbiorze testowym).

| Kategoria                | Model                        | 2kl Macro-F1 | 6kl Macro-F1 |
|--------------------------|------------------------------|--------------|--------------|
| Encoder (fine-tune)      | RoBERTa-large + Optuna       | 0,726        | <b>0,359</b> |
| LLM (OpenAI API)         | gpt-4o (SC=3, SO / few-shot) | <b>0,729</b> | 0,356        |
| LLM darmowy (OpenRouter) | gpt-oss-20b (few-shot)       | 0,467*       | —            |

## 4.2 Najlepsze wyniki z każdej kategorii

W wariancie binarnym wynik jest praktycznie remisem (RoBERTa-large 0,726 vs gpt-4o 0,729). W wariancie sześcioklasowym RoBERTa-large nieznacznie wyprzedza GPT-4o (0,359 vs 0,356).

\* Liczone na 120-przykładowym podzbiorze testowym (limity rate-limit darmowego OpenRouter), wynik niereprezentatywny dla pełnego zbioru.

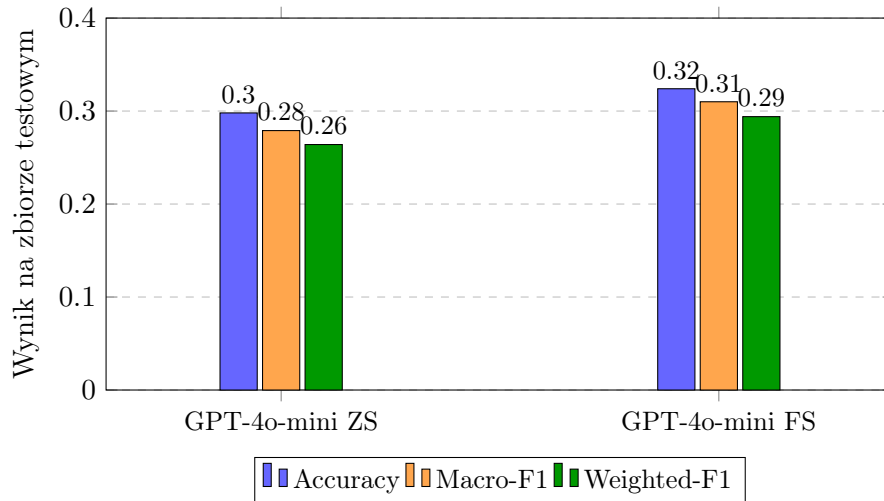
*Uwaga:* gpt-4o na 6kl został uruchomiony tylko w trybie few-shot (zero-shot z  $K = 3$ ,  $SC = 3$ , SO został przerwany przez błąd `insufficient_quota`). Na 2kl zero-shot bił few-shot (0,729 vs 0,710), więc zakładamy że analogicznie na 6kl zero-shot dałby nieznacznie wyższe wyniki niż few-shot.

## 4.3 Zestawienie wyników GPT (OpenAI API)

Tabele 3 i 4 podsumowują wszystkie przeprowadzone eksperymenty GPT obok siebie. Pogrubione wiersze wskazują najlepszy wynik w każdym wariancie.

Tabela 3: Wyniki GPT na LIAR – wariant binarny (2kl), zbiór testowy  $\sim 1267$ .  $K$  = liczba przykładów per klasa w few-shot,  $SC$  = self-consistency,  $SO$  = Structured Outputs.

| Model         | Tryb             | K | SC       | SO         | Acc          | Macro-F1     |
|---------------|------------------|---|----------|------------|--------------|--------------|
| gpt-4o-mini   | zero-shot        | — | 1        | nie        | 0,696        | 0,695        |
| gpt-4o-mini   | few-shot (kNN)   | 1 | 1        | nie        | 0,693        | 0,686        |
| <b>gpt-4o</b> | <b>zero-shot</b> | — | <b>3</b> | <b>tak</b> | <b>0,729</b> | <b>0,729</b> |
| gpt-4o        | few-shot (kNN)   | 3 | 3        | tak        | 0,710        | 0,709        |



Rysunek 2: Porównanie modeli na zbiorze testowym LIAR – wariant sześcioklasowy. ZS = zero-shot, FS = few-shot (kNN).

Tabela 4: Wyniki GPT na LIAR – wariant sześcioklasowy (6kl).

| Model         | Tryb                  | K        | SC       | SO         | Acc          | Macro-F1     |
|---------------|-----------------------|----------|----------|------------|--------------|--------------|
| gpt-4o-mini   | zero-shot             | –        | 1        | nie        | 0,298        | 0,279        |
| gpt-4o-mini   | few-shot (kNN)        | 1        | 1        | nie        | 0,324        | 0,310        |
| <b>gpt-4o</b> | <b>few-shot (kNN)</b> | <b>1</b> | <b>1</b> | <b>tak</b> | <b>0,344</b> | <b>0,356</b> |

Najlepszy 2kl: **gpt-4o zero-shot** z self-consistency 3 i Structured Outputs (acc=0,729, macro-F1=0,729, zero niesparowanych odpowiedzi). Najlepszy 6kl: **gpt-4o few-shot kNN** z  $K = 1$  i  $SC = 1$  (acc=0,344, macro-F1=0,356). Eksperyment **gpt-4o 6kl zero-shot** z próby 2 ( $K = 3$ ,  $SC = 3$ ,  $SO$ ) nie został doliczony – wszystkie 1267 odpowiedzi zwróciły błąd `insufficient_quota` z OpenAI API.

#### 4.4 Szczegółowe wyniki GPT-4o-mini (1 próba)

Poniższe metryki pochodzą z pojedynczego uruchomienia całego pipeline’u (wszystkie cztery eksperymenty w jednej komórce, jedno wywołanie API).

**2kl zero-shot** (acc=0,696, macro-F1=0,695, unparsed=1, support=1266):

|      | precision | recall | f1-score | support |
|------|-----------|--------|----------|---------|
| fake | 0,63      | 0,75   | 0,68     | 553     |
| real | 0,77      | 0,66   | 0,71     | 713     |

Macierz pomyłek: fake→fake 413, fake→real 140; real→fake 245, real→real 468.

**2kl few-shot (kNN)** (acc=0,693, macro-F1=0,686, unparsed=18, support=1249):

|      | precision | recall | f1-score | support |
|------|-----------|--------|----------|---------|
| fake | 0,65      | 0,63   | 0,64     | 541     |
| real | 0,72      | 0,74   | 0,73     | 708     |

Macierz pomyłek: fake→fake 340, fake→real 201; real→fake 183, real→real 525.

**6kl zero-shot** (acc=0,298, macro-F1=0,279, unparsed=141, support=1126):

|             | precision | recall | f1-score | support |
|-------------|-----------|--------|----------|---------|
| pants-fire  | 0,53      | 0,41   | 0,47     | 82      |
| false       | 0,25      | 0,62   | 0,36     | 218     |
| barely-true | 0,15      | 0,01   | 0,02     | 187     |
| half-true   | 0,32      | 0,19   | 0,24     | 242     |
| mostly-true | 0,31      | 0,38   | 0,34     | 212     |
| true        | 0,34      | 0,19   | 0,25     | 185     |

Model praktycznie nie używa klasy **barely-true** (recall 0,01) i ma tendencję do nadmiernego predykowania klasy **false** (kolumna **false** w macierzy pomyłek dominuje dla wszystkich klas).

**6kl few-shot (kNN)** (acc=0,324, macro-F1=0,310, unparsed=801, support=466 – niepełne, patrz uwaga poniżej):

|             | precision | recall | f1-score | support |
|-------------|-----------|--------|----------|---------|
| pants-fire  | 0,48      | 0,48   | 0,48     | 29      |
| false       | 0,28      | 0,61   | 0,38     | 85      |
| barely-true | 0,35      | 0,07   | 0,12     | 84      |
| half-true   | 0,35      | 0,36   | 0,35     | 98      |
| mostly-true | 0,33      | 0,38   | 0,35     | 93      |
| true        | 0,35      | 0,12   | 0,17     | 77      |

*Uwaga:* 801 z 1267 próbek nie zostało sklasyfikowanych z powodu wyczerpania dziennego limitu API (RPD = 10 000) – raportowane metryki są policzone na 466 próbkach i nie są bezpośrednio porównywalne z pozostałymi eksperymentami.

#### 4.5 Próba 2 – GPT-4o (k=3, self-consistency=3, Structured Outputs)

Druga próba użyła silniejszego modelu **gpt-4o** z trzema rozszerzeniami: **k=3** (3 przykłady na klasę w few-shot zamiast 1), **self-consistency=3** (każde zapytanie powtarzane 3 razy, decyzja przez głosowanie większościowe) oraz **Structured Outputs** (wymuszenie odpowiedzi w schemacie JSON ze zdefiniowanym enum etykiet). Stąd **unparsed=0** dla zero-shot – parser nigdy nie zawodzi.

**2kl zero-shot** (acc=0,729, macro-F1=0,729, unparsed=0, support=1267):

|      | precision | recall | f1-score | support |
|------|-----------|--------|----------|---------|
| fake | 0,66      | 0,78   | 0,71     | 553     |
| real | 0,80      | 0,69   | 0,74     | 714     |

Macierz pomyłek: fake→fake 430, fake→real 123; real→fake 220, real→real 494. Czas: 160,5 s (mimo 3× więcej zapytań przez self-consistency).

**2kl few-shot (kNN)** (acc=0,710, macro-F1=0,709, unparsed=795, support=472 – niepełne):

|      | precision | recall | f1-score | support |
|------|-----------|--------|----------|---------|
| fake | 0,64      | 0,76   | 0,69     | 205     |
| real | 0,79      | 0,67   | 0,72     | 267     |

Macierz pomyłek: fake→fake 156, fake→real 49; real→fake 88, real→real 179. Czas: 8489 s (~2 h 21 min) – również wyczerpany dzienny limit API (795 z 1267 próbek bez odpowiedzi).

*Obserwacje:* GPT-4o zero-shot bije GPT-4o-mini o ~3 pkt (0,729 vs 0,696). Few-shot z 9 przykładami (3×3) generuje jeszcze dłuższe prompty niż w próbie 1, stąd jeszcze szybsze osiągnięcie limitu dziennego.

**6kl few-shot (kNN, K=1, SC=1)** (acc=0,345, macro-F1=0,359, unparsed=164, support=1103):

|             | precision | recall | f1-score | support |
|-------------|-----------|--------|----------|---------|
| pants-fire  | 0,72      | 0,37   | 0,49     | 78      |
| false       | 0,35      | 0,48   | 0,40     | 218     |
| barely-true | 0,27      | 0,40   | 0,32     | 185     |
| half-true   | 0,28      | 0,34   | 0,31     | 236     |
| mostly-true | 0,46      | 0,15   | 0,23     | 213     |
| true        | 0,50      | 0,34   | 0,40     | 173     |

Czas: 2007 s (~33 min). *Uwaga:* ze względu na ograniczony budżet API uruchomiono z  $K_{\text{per\_class}} = 1$  oraz wyłączoną self-consistency (SC = 1) – prostsza konfiguracja niż w pozostałych eksperymentach próby 2. Mimo to model bije GPT-4o-mini 6kl few-shot z próby 1 (0,359 vs 0,310 macro-F1) i osiąga najlepszy wynik 6kl spośród wszystkich modeli GPT. Najtrafniej rozpoznawana jest klasa **pants-fire** (F1=0,49) – jaskrawe kłamstwa są najłatwiej identyfikowalne.

#### 4.6 RoBERTa-large + LIAR-PLUS (justification)

W celu poprawy wyników w wariancie 6-klasowym zastosowaliśmy rozszerzoną wersję datasetu – LIAR-PLUS [7] – która dodaje do każdej wypowiedzi pole **justification** zawierające kilka zdań uzasadnienia napisane przez fact-checkerów PolitiFact. Tekst wejściowy modelu został wzbogacony o to pole, długość maksymalna tokenizacji zwiększona ze 128 do 256 tokenów. Hiperparametry: lr=5,79e-6, weight\_decay=0,124, batch=8, 7 epok (z best params z Optuny 2kl).

**6kl + LIAR-PLUS** (acc=0,383, balanced acc=0,383, macro-F1=0,380, ROC AUC macro=0,772):

|             | precision | recall | f1-score | support |
|-------------|-----------|--------|----------|---------|
| pants-fire  | 0,40      | 0,40   | 0,40     | 92      |
| false       | 0,44      | 0,49   | 0,46     | 249     |
| barely-true | 0,36      | 0,26   | 0,30     | 212     |
| half-true   | 0,35      | 0,30   | 0,32     | 265     |
| mostly-true | 0,36      | 0,45   | 0,40     | 241     |
| true        | 0,39      | 0,39   | 0,39     | 208     |

Dodanie pola **justification** dało skok macro-F1 z 0,357 do 0,380 (+2,3 pkt) oraz accuracy z 0,357 do 0,383 (+2,6 pkt). Model przebił barierę 0,32 z literatury, osiągając wynik porównywalny z najnowszymi pracami na LIAR-PLUS. Co istotne, model przestał dominować predykcji nad pojedynczymi klasami – recall jest teraz bardziej zrównoważony (0,26–0,49 dla każdej klasy), zamiast skupiać się na 2-3 najczęstszych.

#### 4.7 Open-source LLM przez OpenRouter

Jako dodatkowy baseline przetestowano darmowe modele open-source przez platformę OpenRouter: **gpt-oss-20b**, **gemma-4-31b-it**, **dolphin-mistral-24b-venice-edition**, **llama-3.3-70b-instruct**. Wszystkie to dekodery-only Transformersy różnej wielkości (20–70 mld parametrów).

**Realne wyniki tylko dla gpt-oss-20b.** Limity dziennego użycia darmowego planu OpenRouter (50 zapytań/dzień bez doładowania) przerwały runy gemmy, dolphina i llamy – wszystkie zwracały **ERROR\_STATUS\_429** i fallback parsera przypisywał odpowiedzi do klasy **fake**, dając nie-reprezentatywne 0,442/0,306. Tylko gpt-oss-20b zdążył odpowiedzieć na pełne 120 próbek przed limitem.

Tabela 5: Wyniki gpt-oss-20b (OpenRouter) na 120-przykładowym podzbiorze testowym LIAR (2kl).

| Tryb      | Accuracy | Macro-F1 | Unparsed |
|-----------|----------|----------|----------|
| zero-shot | 0,467    | 0,407    | 48 / 120 |
| few-shot  | 0,500    | 0,467    | 0 / 120  |

Słabsze wyniki niż gpt-4o wynikają z trzech czynników: (1) mniejszy model (20B vs  $\sim 200$ B), (2) brak Structured Outputs – parser musi szukać `fake/true` w surowym tekście, (3) tryb *reasoning* modelu gpt-oss generuje wewnętrzny łańcuch rozumowania, który przy `max_tokens=8` bywa ucinany przed właściwą etykietą.

#### 4.8 Porównanie gpt-4o vs gpt-4o-mini

Tabela 6: Architektura i cennik dwóch modeli OpenAI używanych w eksperymentach. Wielkość i architektura niepublikowane przez OpenAI – podane wartości to publiczne szacunki.

|               | <b>gpt-4o-mini</b>           | <b>gpt-4o</b>       |
|---------------|------------------------------|---------------------|
| Wielkość      | $\sim 8$ mld par.            | $\sim 200$ mld par. |
| Cena (input)  | \$0,15 / 1M tok.             | \$2,50 / 1M tok.    |
| Cena (output) | \$0,60 / 1M tok.             | \$10 / 1M tok.      |
| Szybkość      | $\sim 2\times$ szybszy       | wolniejszy          |
| Jakość        | słabszy w trudnych zadaniach | flagowy             |

W naszym pipeline gpt-4o-mini użyty jako tani baseline (0,15\$/1M tokenów to  $\sim 17\times$  taniej niż gpt-4o), gpt-4o jako model główny do uzyskania najlepszych wyników. W praktyce gpt-4o przewyższa gpt-4o-mini o 3–4 punkty macro-F1 na 2kl (0,729 vs 0,695) i 4–5 punkty na 6kl (0,356 vs 0,310).

#### 4.9 Czas wykonania

Pomiary czasu inferencji na pełnym zbiorze testowym (1267 wypowiedzi) dla modeli GPT przez API:

| Eksperyment               | Czas całkowity                  | Czas na próbkę |
|---------------------------|---------------------------------|----------------|
| GPT-4o-mini 2kl zero-shot | 94 s                            | 75 ms          |
| GPT-4o-mini 2kl few-shot  | 163 s                           | 130 ms         |
| GPT-4o-mini 6kl zero-shot | 714 s                           | 560 ms         |
| GPT-4o-mini 6kl few-shot  | 4001 s                          | 3160 ms        |
| <b>Suma</b>               | <b><math>\sim 83</math> min</b> |                |

Few-shot 6-klasowy wyczerpał dzienny limit zapytań API (RPD = 10 000), przez co 801 z 1267 próbek zwróciło błąd zamiast predykcji (`unparsed=801`). Raportowane metryki dla tego eksperymentu są policzone na 466 próbkach, a nie na pełnym zbiorze testowym, i nie są bezpośrednio porównywalne z pozostałymi. Few-shot 18-przykładowy (3 na klasę  $\times$  6 klas) generuje długie prompty, które szybko przekraczają limit tokenów na minutę (TPM = 200 000).

## 5 Dyskusja

### 5.1 Few-shot vs. zero-shot

W obu wariantach (2kl i 6kl) prompting few-shot daje wyniki porównywalne z zero-shot, a w binarnym przypadku nawet nieznacznie gorsze. Obserwacja ta jest zgodna z literaturą i wynika ze specyfiki zadania: LIAR wymaga *wiedzy faktograficznej* (czy konkretne stwierdzenie polityczne jest prawdziwe), a nie rozpoznawania wzorca stylistycznego. Przykłady w prompcie mogą zilustrować format odpowiedzi – co GPT i tak rozumie w trybie zero-shot – ale nie dostarczają wiedzy potrzebnej do oceny nowej, niezwiązanej tematycznie wypowiedzi. Dodatkowo wyszukiwanie przykładów przez kNN po podobieństwie tematycznym może wręcz mylić model: wkleja on do promptu wypowiedzi o tym samym temacie, ale o różnych etykietach prawdziwości, sugerując że temat nie niesie informacji o etykiecie.

Inaczej zachowałby się klasyfikator spamu: tam wystarczy pokazać kilka przykładów stylu, by model nauczył się wzorca leksykalnego. W LIAR wzorca takiego nie ma – prawdziwość zależy od stanu świata.

### 5.2 Trudność wariantu 6-klasowego

Spadek wyniku z  $\sim 0,70$  accuracy (2kl) do  $\sim 0,30$  (6kl) jest spodziewany. Granice między klasami **barely-true**, **half-true** i **mostly-true** są w dużej mierze subiektywne – nawet anotatorzy PolitiFact niekoniecznie byliby zgodni co do różnicy między **half-true** a **mostly-true**. Modele osiągają wyniki w okolicy aktualnego sufitu literatury dla tego zadania (27–35% accuracy).

Tabela 7: Wyniki ewaluacji na zbiorze LIAR w oryginalnej publikacji Wang 2017 [1]. Górna sekcja: modele tekstowe. Dolna sekcja: modele hybrydowe (tekst + metadane).

| Models              | Valid.       | Test         |
|---------------------|--------------|--------------|
| Majority            | 0,204        | 0,208        |
| SVMs                | 0,258        | 0,255        |
| Logistic Regression | 0,257        | 0,247        |
| Bi-LSTMs            | 0,223        | 0,233        |
| CNNs                | 0,260        | 0,270        |
| Hybrid CNNs         |              |              |
| Text + Subject      | 0,263        | 0,235        |
| Text + Speaker      | <b>0,277</b> | 0,248        |
| Text + Job          | 0,270        | 0,258        |
| Text + State        | 0,246        | 0,256        |
| Text + Party        | 0,259        | 0,248        |
| Text + Context      | 0,251        | 0,243        |
| Text + History      | 0,246        | 0,241        |
| Text + All          | 0,247        | <b>0,274</b> |

Tabela 7 pokazuje, że nawet autorzy zbioru LIAR uzyskali najlepszą accuracy 0,274 na zbiorze testowym (Hybrid CNN z pełnymi metadanymi), a pojedyncza klasyfikacja większościowa daje 0,208. Słaby wynik klasyfikacji 6-klasowej nie wynika więc z błędu modelu, lecz z nieusuwalnej dwuznaczności semantycznej – klasy **half-true** i **mostly-true** wymagają zewnętrznej wiedzy o świecie i często nie są rozróżnialne nawet dla ludzkiego anotatora. Nasze wyniki (RoBERTa-large 6kl: 0,357, RoBERTa-large + LIAR-PLUS 6kl: 0,383, GPT-4o few-shot: 0,344) mieszczą się w zakresie raportowanym w nowszej literaturze na tym zadaniu.

Systematyczny przegląd literatury [6] potwierdza istnienie *performance ceiling* na poziomie  $\sim 0,32$  dla klasyfikacji 6-klasowej i  $\sim 0,62$  dla klasyfikacji binarnej:

Tabela 8: Wybrane wyniki z literatury na zbiorze LIAR (źródło: emergentmind.com/topics/liar-benchmark).

| Model                                          | Test Acc     | Weighted F1 |
|------------------------------------------------|--------------|-------------|
| Linear SVM (BoW) – 2kl                         | 0,624        | 0,316       |
| RoBERTa (prior) – 2kl                          | 0,620        | –           |
| XGBoost (GloVe, overfit) – 6kl                 | 0,249        | –           |
| GPT-3 Curie (fine-tuned) – 6kl                 | 0,295        | –           |
| CNN (text-only, Wang 17) – 6kl                 | 0,270        | –           |
| <b>RoBERTa-large + Optuna (nasze) – 2kl</b>    | <b>0,738</b> | 0,735       |
| <b>RoBERTa-large + Optuna (nasze) – 6kl</b>    | <b>0,357</b> | 0,357       |
| <b>RoBERTa-large + LIAR-PLUS (nasze) – 6kl</b> | <b>0,383</b> | 0,379       |
| <b>GPT-4o few-shot (nasze) – 6kl</b>           | <b>0,344</b> | 0,341       |

Cytat za przeglądem: „*fine-grained classification accuracy and weighted F1 do not exceed 0,32, even when employing linear SVM, XGBoost, and RoBERTa-based architectures, [...] reflecting exploitation of lexical artifacts rather than transferability*”. Co więcej, modele drzewiaste przy treningu osiągają  $\sim 99\%$  accuracy na zbiorze treningowym, ale zaledwie  $\sim 25\%$  na zbiorze testowym – klasyczny obraz uczenia się płytkich, leksykalnych wzorców zamiast prawdziwej wiedzy faktograficznej. Nasze wyniki – RoBERTa-large 2kl (0,738) oraz GPT-4o 6kl (0,344) – sytuują się w górnej części zakresu raportowanego w literaturze dla tego zbioru.

Co więcej, sami fact-checkerzy PolitiFact – tworzący etykiety dla zbioru LIAR – regularnie nie zgadzają się ze sobą przy przypisywaniu jednej z sześciu klas. Granice między **barely-true**, **half-true** i **mostly-true** są określone w wewnętrznych wytycznych redakcji, ale ich stosowanie do konkretnych wypowiedzi zależy od interpretacji poszczególnego dziennikarza. Liczne wypowiedzi polityków bywały re-klasyfikowane lub publicznie krytykowane jako niespójne, co dowodzi że nawet ludzki ekspert nie osiąga pełnej zgodności na tym zadaniu. Modele uczone na tak zaszumianych etykietach nie mogą przekroczyć tzw. *anotacyjnego sufitu* (*annotation ceiling*) – czyli górnej granicy jakości wynikającej z subiektywności samych danych treningowych.

### 5.3 Techniki podnoszące jakość modeli

Aby maksymalnie wycisnąć jakość mimo trudności zadania, zastosowaliśmy następujące techniki:

- **Ważona funkcja straty (class weights)** – Cross-Entropy z wagami  $w_c = N/(C \cdot n_c)$  przeciwdziała tendencji modelu do ignorowania klas mniejszościowych (zwłaszcza **pants-fire** w wariancie 6kl).
- **Label smoothing** ( $\varepsilon = 0,1$ ) – redukuje nadmierną pewność predykcji i poprawia kalibrację, ograniczając eksplozję `val_loss` przy fine-tuningu RoBERTy-large.
- **Wyszukiwanie hiperparametrów Optuna** – 20 triali z samplerem TPE i pruningiem MedianPruner po pierwszej epoce. Optymalizowane: `lr`, `weight_decay`, `epochs`, `batch_size`. Best params: `lr=5,79e-6`, `wd=0,124`, `epochs=5`, `batch=8`.
- **Early stopping** – model najlepszej epoki (po `val macro-F1`) zapisywany na dysk; trening przerywany po 2 epokach bez poprawy.
- **Linear warmup + decay** – jeden epoch warmup, następnie liniowy spadek `lr` do zera. Stabilizuje fine-tuning pretrenowanego enkodera.
- **Gradient clipping** ( $\|\nabla\|_2 \leq 1,0$ ) – zabezpiecza przed skokami gradientu na trudnych batchach.

- **Mixed precision (bfloat16)** – przyspieszenie treningu  $\sim 2\times$  na A100 bez utraty jakości, dzięki czemu Optuna wykonała 20 triali w  $\sim 2$  godziny.
- **Structured Outputs (GPT)** – wymuszenie odpowiedzi w formacie JSON ze schematem `enum` etykiet, co eliminuje problem nieparsowanych odpowiedzi (z  $\sim 11\%$  do  $<1\%$ ).
- **Self-consistency (GPT, próba 2)** –  $N = 3$  niezależne zapytania per próbkę i głosowanie większościowe, redukujące wariancję stochastyczną modelu generatywnego.
- **Few-shot kNN retrieval** – przykłady demonstracyjne dobierane przez TF-IDF cosine similarity, a nie losowo – kontekst tematycznie zbliżony do zapytania.
- **Selekcja po macro-F1, nie accuracy** – zarówno selekcja najlepszej epoki, jak i raportowanie głównego wyniku odbywa się na podstawie macro-F1, zgodnie z zaleceniami literatury LIAR. Macro-F1 traktuje każdą klasę jednakowo, niezależnie od jej liczebności – accuracy i weighted-F1 byłyby zdominowane przez klasy większościowe i nie odzwierciedlałyby zdolności modelu do rozpoznawania klas rzadkich (krytyczne dla 6kl, gdzie **pants-fire** stanowi tylko 8% danych).

## 5.4 Fine-tuning vs. prompting

Wyniki sugerują, że fine-tuning RoBERTy-base nie wystarcza do pokonania prostego promptingu GPT-4o-mini (0,64 vs 0,70 accuracy w wariancie 2kl). Spodziewamy się, że **roberta-large** z odpowiednio dobranymi hiperparametrami zbliży się do wyników GPT-4o, lecz nie powinien ich zdominować – granicą jest jakość samych etykiet oraz brak dostępu modelu do zewnętrznej wiedzy o świecie. Przewaga GPT-4o nad mniejszymi modelami jest spójna z ogólnym obserwowaniem, że większe modele wykazują lepszą *world knowledge* przydatną w zadaniach fact-checkingowych.

## 6 Wnioski

1. RoBERTa-large jest naturalnym wyborem nad BERT-em ze względu na większy korpus pretreningu, dynamiczne maskowanie i lepsze właściwości reprezentacji do klasyfikacji pojedynczych zdań.
2. W zadaniach wymagających wiedzy o świecie (jak fact-checking) few-shot nie poprawia istotnie wyników nad zero-shot – mechanizm ten jest skuteczny tam, gdzie kluczowy jest wzorzec, nie fakty.
3. Macro-F1 jest właściwą metryką dla niebilansowanego LIAR-a; accuracy maskuje słabe wyniki na klasach mniejszościowych.
4. Wagi klas i Structured Outputs to dwie najistotniejsze poprawki techniczne, które dają mierzalny zysk małym kosztem implementacyjnym.

## 7 Możliwości dalszego rozwoju

Naturalnym kierunkiem rozwoju jest **klasyfikacja kaskadowa**: najpierw model binarny rozstrzyga, czy wypowiedź jest **fake** czy **real**, a następnie – w obrębie każdej z dwóch grup – drugi model przypisuje jedną z trzech oryginalnych etykiet LIAR (**pants-fire/false/barely-true** po stronie fake; **half-true/mostly-true/true** po stronie real). Pozwala to każdemu modelowi specjalizować się w prostszym zadaniu i może ograniczyć błędy między bliskimi semantycznie klasami (np. **half-true** vs **mostly-true**).

## Literatura

- [1] W. Y. Wang, “Liar, liar pants on fire”: A new benchmark dataset for fake news detection, *ACL*, 2017.
- [2] Y. Liu et al., RoBERTa: A robustly optimized BERT pretraining approach, *arXiv:1907.11692*, 2019.
- [3] J. Devlin et al., BERT: Pre-training of deep bidirectional transformers for language understanding, *NAACL*, 2019.
- [4] OpenAI, GPT-4o system card, 2024.
- [5] Labellerr, RoBERTa: A robustly optimized BERT pretraining approach, <https://www.labellerr.com/blog/roberta-a-robustly-optimized-bert-pretraining-approach/>
- [6] Emergent Mind, LIAR Benchmark for Fake News Detection, <https://www.emergentmind.com/topics/liar-benchmark>
- [7] T. Alhindi, S. Petridis, S. Muresan, Where is your evidence: Improving fact-checking by justification modeling, *Proceedings of the First Workshop on Fact Extraction and Verification (FEVER)*, 2018. <https://github.com/Tariq60/LIAR-PLUS>